CHANDIGARH UNIVERSITY

PYTHON PROGRAMMING

MINI PROJECT

ATM SIMULATOR USING PYTHON

SUBMITTED BY

Name:
Jovanpreet Kaur

UID:
25BCD10052

Section: 25BCD-1(B)

Branch: BCA Data Science

SUBMITTED TO

Name:
Mr. Ishant

Designation:
Professor

Index

S.No.	Topic	Page
1	Introduction	3
2	Problem Statement	4
3	Objectives of the Project	5
4	Scope & Limitations	6
5	Tools and Technologies Used	7
6	System Design	8
7	Source Code	10
8	Code Explanation	12
9	Sample Output	14
10	Testing and Results	16
11	Advantages	17
12	Future Scope	18
13	Conclusion	19
14	GitHub Repository	20
15	References	21

Introduction

An Automated Teller Machine (ATM) is an electronic banking machine that allows customers to perform basic banking transactions such as checking the balance, depositing money and withdrawing cash without visiting a bank branch. Before any transaction is allowed, the machine verifies the customer with a secret Personal Identification Number (PIN).

This mini project, **ATM Simulator Using Python**, is a console-based program that imitates the working of a real ATM. It has been developed as part of the Python Programming course at Chandigarh University to apply the basic concepts learned in class, namely variables, data types, conditional statements, loops, functions, lists and exception handling, in a practical and easy to understand application.

When the program starts, the user must enter the correct PIN within three attempts. After a successful login, a menu is displayed from which the user can check the balance, deposit money, withdraw money, view a mini statement of recent transactions, or exit. Every input is validated so that wrong or unexpected input never crashes the program.

Python was chosen because of its simple and readable syntax, which lets beginners concentrate on logic and problem solving rather than on complicated language rules. The complete project is written in a single file and needs no external library.

Problem Statement

Banking transactions such as deposits, withdrawals and balance enquiries must be carried out accurately and only after verifying the identity of the account holder. Students who are learning programming often find it difficult to connect abstract concepts like loops, conditions and functions with a real-life problem.

The problem addressed in this project is: to design and develop a console-based ATM simulator in Python that

- authenticates the user with a PIN and allows only a limited number of attempts;
- lets the authenticated user check the balance, deposit money and withdraw money;
- prevents invalid transactions such as withdrawing more than the available balance or entering negative or non-numeric amounts;
- keeps a record of the transactions and displays a mini statement;
- keeps running through a menu until the user chooses to exit.

The simulator provides a safe environment in which these operations can be tried without handling real money or connecting to a real bank.

Objectives of the Project

The main objectives of this project are:

- To understand and apply the basic building blocks of Python: variables, input/output, operators, conditions and loops.
- To use functions to divide a problem into small, reusable and easy to manage parts.
- To implement PIN-based authentication with a limited number of attempts.
- To perform balance enquiry, deposit and withdrawal operations correctly.
- To handle invalid user input using **try/except** blocks and validation checks.
- To store and display recent transactions using a list.
- To develop a menu-driven program that is simple, user friendly and easy to extend.
- To gain hands-on experience in writing, testing and documenting a complete mini project.

Scope & Limitations

Scope

- Demonstrates how a real ATM works in a simple, text-based form.
- Useful as a learning project for beginners in Python.
- Covers the most common ATM operations: balance enquiry, deposit, withdrawal and mini statement.
- Can be extended into a larger banking application by adding more features (see Future Scope).

Limitations

- The program works for a single account with a fixed PIN written in the code.
- Data is stored only in memory; the balance and the transaction history are lost when the program is closed.
- The PIN is stored as plain text and is visible while typing, so the program is not secure for real-world use.
- It runs only in the console; there is no graphical user interface.
- The mini statement stores only the type and amount of a transaction, not the date and time.
- It is a simulation and is not connected to any real bank or database.

Tools and Technologies Used

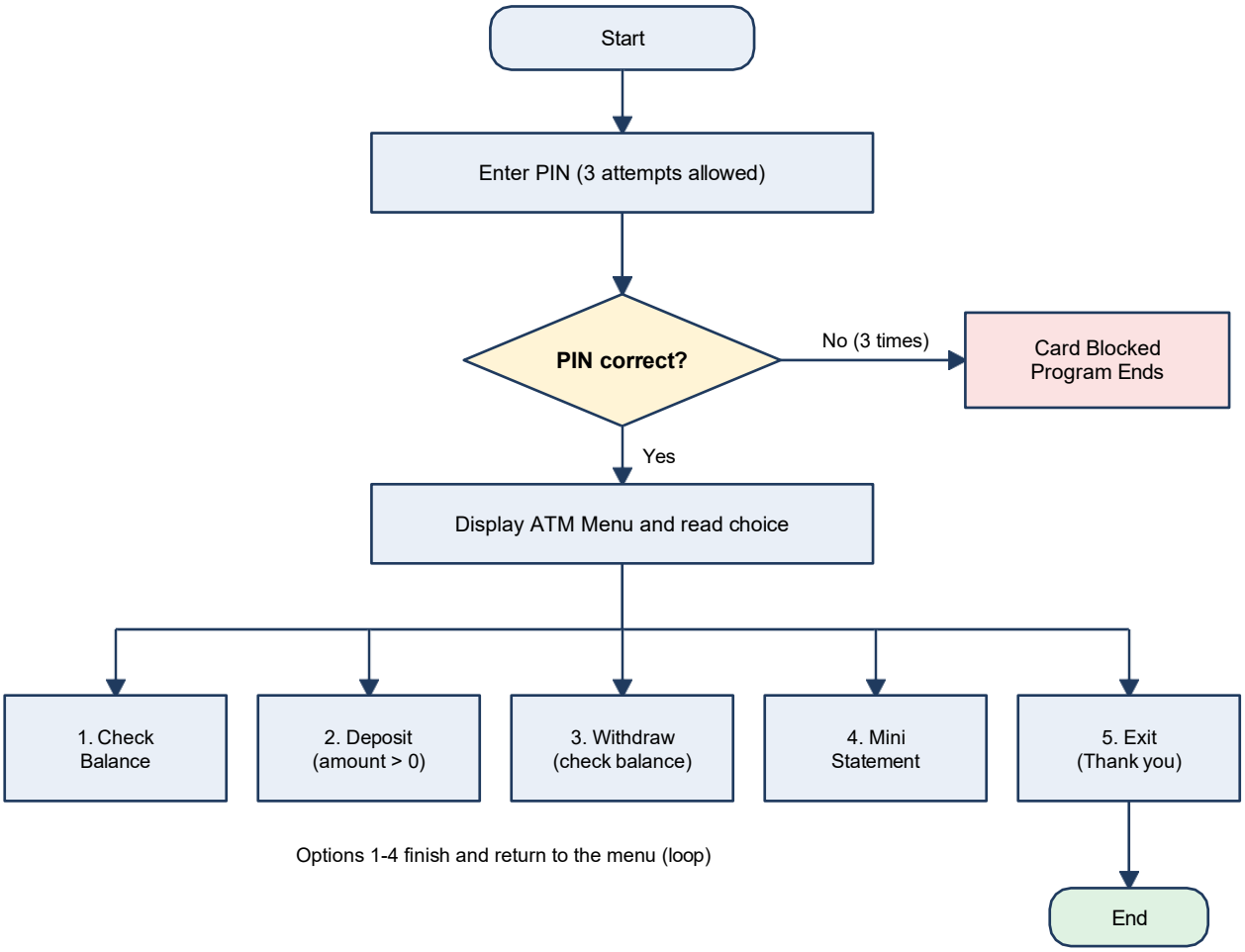
Category	Tool / Technology	Purpose
Programming language	Python 3	Writing the complete program logic
Code editor / IDE	VS Code, IDLE or PyCharm	Writing, running and debugging the code
User interface	Command line / console	Taking input and displaying output
Libraries	None (only built-in features)	The project runs on a plain Python installation
Version control	Git and GitHub	Storing and sharing the project
Documentation	PDF report	Presenting the project to the faculty

Python concepts used

Concept	Where it is used
Variables and data types (int, float, str, list)	`CORRECT_PIN`, `balance`, `transactions`
Functions	Every ATM feature is a separate function
while loop	PIN attempts and the main menu
if / elif / else	Menu choice and balance checks
try / except	Handling non-numeric input
global keyword	Changing the balance inside functions
Lists and slicing	Transaction history, last 5 entries
f-strings	Formatting messages and amounts

System Design

The program follows a simple modular design. Each feature is written as a separate function and the `main()` function controls the flow of the program. The flowchart below shows the working of the system.



Modules (functions) of the system

Function	Purpose
check_pin()	Gives the user 3 attempts to enter the correct PIN; returns True or False.
get_amount(message)	Reads an amount and validates that it is a positive number.
show_menu()	Prints the ATM menu.
check_balance()	Displays the current balance.
deposit()	Adds money to the balance and records the transaction.
withdraw()	Deducts money if the balance is sufficient and records the transaction.
mini_statement()	Shows the last 5 transactions.
main()	Controls the whole program: login, menu loop and exit.

Data used by the program

Name	Type	Description
CORRECT_PIN	int	The PIN of the account (1234).
balance	float	Current account balance (starts at Rs. 10000).
transactions	list	Text records of deposits and withdrawals.

Source Code

File name: `atm_simulator.py`

```
"""
Simple ATM Simulator
Concepts used: variables, functions, loops, conditions, input validation
"""

# ---- Account details (you can change these) ----
CORRECT_PIN = 1234
balance = 10000.0
transactions = []  # keeps a history of everything done


def check_pin():
    """Give the user 3 attempts to enter the correct PIN."""
    attempts = 3
    while attempts > 0:
        try:
            pin = int(input("Enter your 4-digit PIN: "))
        except ValueError:
            print("PIN must be numbers only.")
            attempts -= 1
            continue

        if pin == CORRECT_PIN:
            print("\nLogin successful!\n")
            return True

        attempts -= 1
        print(f"Wrong PIN. Attempts left: {attempts}")

    print("Card blocked. Too many wrong attempts.")
    return False


def get_amount(message):
    """Ask for an amount and make sure it is a valid positive number."""
    try:
        amount = float(input(message))
    except ValueError:
        print("Please enter a valid number.")
        return None

    if amount <= 0:
        print("Amount must be greater than zero.")
        return None
    return amount


def show_menu():
    print("\n===== ATM MENU =====")
    print("1. Check Balance")
    print("2. Deposit")
    print("3. Withdraw")
    print("4. Mini Statement")
    print("5. Exit")


def check_balance():
    print(f"Your current balance is: Rs. {balance:.2f}")


def deposit():
    global balance
    amount = get_amount("Enter amount to deposit: ")
    if amount is None:
        return
    balance += amount
    transactions.append(f"Deposited  Rs. {amount:.2f}")
```

```
    print(f"Rs. {amount:.2f} deposited successfully.")
    check_balance()


def withdraw():
    global balance
    amount = get_amount("Enter amount to withdraw: ")
    if amount is None:
        return
    if amount > balance:
        print("Insufficient balance!")
        return
    balance -= amount
    transactions.append(f"Withdrew   Rs. {amount:.2f}")
    print(f"Please collect your cash: Rs. {amount:.2f}")
    check_balance()


def mini_statement():
    print("\n--- Last 5 Transactions ---")
    if not transactions:
        print("No transactions yet.")
    else:
        for t in transactions[-5:]:
            print(t)


def main():
    print("Welcome to Python ATM")
    if not check_pin():
        return

    while True:
        show_menu()
        choice = input("Choose an option (1-5): ")

        if choice == "1":
            check_balance()
        elif choice == "2":
            deposit()
        elif choice == "3":
            withdraw()
        elif choice == "4":
            mini_statement()
        elif choice == "5":
            print("Thank you for using our ATM. Goodbye!")
            break
        else:
            print("Invalid choice. Please select 1-5.")


main()
```

Code Explanation

1. Global data (lines 6-9)

`CORRECT_PIN` stores the PIN (1234), `balance` stores the opening balance of Rs. 10000, and the list `transactions` collects a text record of every deposit and withdrawal.

2. check_pin()

A `while` loop runs while attempts are left (3 in total). The input is converted with `int()` inside a `try` block, so letters raise a ValueError which is caught and counted as a failed attempt. If the PIN matches, the function returns True. After three failures it prints the 'Card blocked' message and returns False.

3. get_amount(message)

Reads an amount with `float()`. If the text is not a number or the amount is zero or negative, it prints a message and returns `None`, which tells the calling function to stop. Otherwise, it returns the valid amount. Because deposit and withdraw both use it, the validation code is written only once.

4. show_menu() and check_balance()

`show_menu()` prints the five options. `check_balance()` prints the balance using an f-string with two decimal places (`:.2f`).

5. deposit()

Uses the `global` keyword so the function can change the global `balance`. After getting a valid amount, it adds it to the balance, appends a record to `transactions`, and shows the new balance.

6. withdraw()

Gets a valid amount and then checks whether it is more than the balance. If so, it prints 'Insufficient balance!' and returns without changing anything. Otherwise, it subtracts the amount, records the transaction and shows the new balance.

7. mini_statement()

If the list is empty it prints 'No transactions yet.' Otherwise, it uses the slice `transactions[-5:]` to print only the last five records.

8. main()

First calls `check_pin()`; if login fails, the program ends. After a successful login, an infinite `while True` loop shows the menu and reads the choice. An `if/elif/else` chain calls the matching function. Choosing 5 prints a goodbye message and uses `break` to leave the loop; any other value prints 'Invalid choice'.

9. Program start (last line)

`main()` is called at the end of the file to start the program.

Sample Output

Output 1: Successful login, balance check, deposit, withdrawal and mini statement

```
Welcome to Python ATM
Enter your 4-digit PIN: 1234

Login successful!


===== ATM MENU =====
1. Check Balance
2. Deposit
3. Withdraw
4. Mini Statement
5. Exit
Choose an option (1-5): 1
Your current balance is: Rs. 10000.00

===== ATM MENU =====
1. Check Balance
2. Deposit
3. Withdraw
4. Mini Statement
5. Exit
Choose an option (1-5): 2
Enter amount to deposit: 2000
Rs. 2000.00 deposited successfully.
Your current balance is: Rs. 12000.00

===== ATM MENU =====
1. Check Balance
2. Deposit
3. Withdraw
4. Mini Statement
5. Exit
Choose an option (1-5): 3
Enter amount to withdraw: 500
Please collect your cash: Rs. 500.00
Your current balance is: Rs. 11500.00

===== ATM MENU =====
1. Check Balance
2. Deposit
3. Withdraw
4. Mini Statement
5. Exit
Choose an option (1-5): 4

--- Last 5 Transactions ---
Deposited  Rs. 2000.00
Withdrew   Rs. 500.00

===== ATM MENU =====
1. Check Balance
2. Deposit
3. Withdraw
4. Mini Statement
5. Exit
Choose an option (1-5): 5
Thank you for using our ATM. Goodbye!
```

Output 2: Wrong PIN entered three times

```
Welcome to Python ATM
Enter your 4-digit PIN: 1111
Wrong PIN. Attempts left: 2
Enter your 4-digit PIN: abcd
PIN must be numbers only.
Enter your 4-digit PIN: 9999
Wrong PIN. Attempts left: 0
Card blocked. Too many wrong attempts.
```

Output 3: Invalid inputs (insufficient balance, negative amount, text input, wrong menu choice)

```
Welcome to Python ATM
Enter your 4-digit PIN: 1234

Login successful!


===== ATM MENU =====
1. Check Balance
2. Deposit
3. Withdraw
4. Mini Statement
5. Exit
Choose an option (1-5): 3
Enter amount to withdraw: 50000
Insufficient balance!

===== ATM MENU =====
1. Check Balance
2. Deposit
3. Withdraw
4. Mini Statement
5. Exit
Choose an option (1-5): 3
Enter amount to withdraw: -5
Amount must be greater than zero.

===== ATM MENU =====
1. Check Balance
2. Deposit
3. Withdraw
4. Mini Statement
5. Exit
Choose an option (1-5): 3
Enter amount to withdraw: abc
Please enter a valid number.

===== ATM MENU =====
1. Check Balance
2. Deposit
3. Withdraw
4. Mini Statement
5. Exit
Choose an option (1-5): 9
Invalid choice. Please select 1-5.

===== ATM MENU =====
1. Check Balance
2. Deposit
3. Withdraw
4. Mini Statement
5. Exit
Choose an option (1-5): 5
Thank you for using our ATM. Goodbye!
```

Testing and Results

The program was run with different inputs to check that each feature works correctly and that invalid input is handled without errors. The results are given below (starting balance Rs. 10000).

No.	Test case	Expected result	Actual result	Status
1	Enter correct PIN 1234	Login successful	Login successful	Pass
2	Enter PIN 1111, abcd, 9999	Card blocked after 3 attempts	Wrong PIN, 'PIN must be numbers only', then 'Card blocked'	Pass
3	Check balance	Rs. 10000.00	Rs. 10000.00	Pass
4	Deposit 2000	Balance becomes Rs. 12000.00	Rs. 12000.00	Pass
5	Withdraw 500	Balance becomes Rs. 11500.00	Rs. 11500.00	Pass
6	Withdraw 50000 (more than balance)	Insufficient balance message	Insufficient balance!	Pass
7	Withdraw -5	Amount rejected	Amount must be greater than zero.	Pass
8	Withdraw 'abc'	Invalid number message	Please enter a valid number.	Pass
9	Mini statement	Shows the last transactions	Deposited 2000.00, Withdrew 500.00	Pass
10	Menu choice 9	Invalid choice message	Invalid choice. Please select 1-5.	Pass
11	Menu choice 5	Program exits	Thank you message, program ends	Pass

Result: All 11 test cases passed. The program handled wrong PINs, invalid amounts, insufficient balance and wrong menu choices without crashing.

Advantages

- Simple, readable code that is easy for beginners to understand.
- Menu-driven, so the user can easily operate the program.
- Modular design: every feature is a separate function, which makes it easy to test, change and extend.
- PIN verification with limited attempts adds a basic level of security.
- Input validation prevents crashes caused by wrong or unexpected input.
- Prevents overdrawing by checking the balance before every withdrawal.
- Keeps a record of transactions that can be viewed as a mini statement.
- No external library or internet connection is required; it runs on any system with Python.
- Safe environment to practise banking logic without real money.

Future Scope

The project can be improved in many ways:

- **Permanent storage:** save the balance and transactions in a file or a database (such as SQLite) so that data is not lost on exit.
- **Multiple accounts:** support many users with different account numbers and PINs using dictionaries or classes.
- **Change PIN option:** allow the user to change the PIN after login.
- **Date and time:** store a timestamp with every transaction using the `datetime` module.
- **Better security:** hide the PIN while typing and store it in encrypted or hashed form.
- **Limits:** add a daily withdrawal limit and a minimum balance rule.
- **Graphical interface:** build a window-based version using Tkinter.
- **Web version:** convert it into a web application using Flask or Django.
- **Object-oriented design:** rewrite the program using classes such as `Account` and `ATM`.

Conclusion

The ATM Simulator project successfully demonstrates how the basic concepts of Python can be used to build a useful, menu-driven application. The program verifies the user with a PIN, performs balance enquiry, deposit and withdrawal, shows a mini statement and handles invalid input without crashing. All the test cases given in this report produced the expected results.

While developing this project, I learned how to break a problem into functions, use loops and conditions to control the flow of a program, handle exceptions, and validate user input. The project also showed me the importance of testing a program with both correct and incorrect inputs.

Although this is a simple simulation with limitations such as a single account and no permanent storage, it forms a strong base. With the improvements listed in the future scope, it can be developed into a more complete banking application.

GitHub Repository

https://github.com/kaurjovan29/ATM-Simulator-using-Python

Repository contents

- `atm_simulator.py` - complete source code of the project
- `README.md` - short description and instructions to run the program
- Project report (PDF)

References

1 Python Software Foundation, *Python 3 Documentation*. https://docs.python.org/3/

2 W3Schools, *Python Tutorial*. https://www.w3schools.com/python/

3 GeeksforGeeks, *Python Programming Language*. https://www.geeksforgeeks.org/python-programming-language/

4 Real Python, *Python Tutorials*. https://realpython.com/

5 Matthes, E., *Python Crash Course*, No Starch Press.

6 Class lectures and notes of Python Programming, Chandigarh University.